

Aula 13 — Como saber se ainda funciona?

Como verificar automaticamente que uma mudança preservou comportamentos que já funcionavam?

A COLABORAÇÃO QUE VAMOS MUDAR

Pedido já funciona com entrega normal

```
Pedido pedido = new Pedido();  
pedido.fechar(new EntregaNormal());
```

Depois, queremos aceitar outras formas de entrega.

Como preservar o que já estava correto?

UMA REGRA CONHECIDA

Duas unidades de teclado

```
Produto teclado = new Produto("Teclado", 150.0);  
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 2);  
  
System.out.println(pedido.calcularTotal());
```

Como você confere que o resultado está correto?

ATIVIDADE

ANTES DA FERRAMENTA

Preveja e compare

O teclado custa **150.0**. O pedido contém **duas unidades**.

1. Que valor deve aparecer?
2. Quem compara esse valor com o resultado impresso?

O PRIMEIRO RESULTADO

O console mostra um valor

```
300.0
```

Nós sabemos que é o esperado porque fizemos a conta.

O programa ainda não conhece essa expectativa.

ESCREVA A EXPECTATIVA

O programa pode fazer a comparação

```
System.out.println(  
    pedido.calcularTotal() == 300.0  
);
```

```
true
```

Melhorou. O que ainda precisamos fazer?

Esperado e observado no código

```
double esperado = 300.0;  
double observado = pedido.calcularTotal();  
  
if (observado != esperado) {  
    System.out.println("ERRO: esperado " + esperado  
        + ", obtido " + observado);  
}
```

Agora o programa aponta a divergência.

ATIVIDADE

SIGA O DESVIO

O que aparece no console?

Com `esperado = 300.0` :

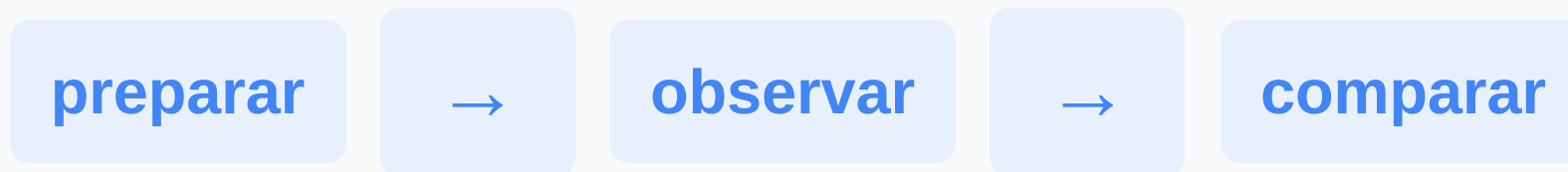
1. se o método devolver `300.0` ;
2. se o método devolver `150.0` .

O que mudaria se tivéssemos **trinta regras**?

A NECESSIDADE SURTIU

Muitas verificações, uma mesma estrutura

Para cada regra, queremos:



Também queremos executar tudo e saber **qual cenário falhou**.

CONCEITO-CHAVE

O QUE ESTAMOS AUTOMATIZANDO

Conceito-chave — expectativa executável

Um teste prepara um cenário, observa uma operação e compara o resultado com uma expectativa definida.

A expectativa fica escrita no código.

JUnit reúne verificações em métodos

```
@Test
void calculaTotalDeUmItem() {
    Produto teclado = new Produto("Teclado", 150.0);
    Pedido pedido = new Pedido();
    pedido.adicionarItem(teclado, 2);

    assertEquals(300.0, pedido.calcularTotal());
}
```

O cenário da Unidade 01 continua o mesmo.

JAVA EM FOCO

O MÍNIMO PARA LER O TESTE

Java em foco — JUnit básico

`@Test` marca uma verificação para o executor.

`assertEquals(esperado, observado)` compara os valores.

Se diferirem, o teste falha e JUnit mostra qual método falhou.

O que precisamos importar

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class PedidoTest {
    @Test
    void calculaTotalDeUmItem() {
        // cenário, ação e verificação
    }
}
```

O executor chama o método de teste. Não usamos `main` para isso.

ATIVIDADE

LEIA ANTES DE EXECUTAR

Que história o teste conta?

```
Produto teclado = new Produto("Teclado", 150.0);  
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 2);  
  
assertEquals(300.0, pedido.calcularTotal());
```

Qual é o cenário? Qual operação observamos? O que esperamos?

TRÊS MOVIMENTOS NO MESMO TESTE

Prepare → execute → verifique

```
Produto teclado = new Produto("Teclado", 150.0);  
Pedido pedido = new Pedido();  
pedido.adicionarItem(teclado, 2);  
  
double total = pedido.calcularTotal();  
  
assertEquals(300.0, total);
```

Arrange → Act → Assert é um nome útil, não uma exigência de comentários.

UM NOME TAMBÉM INFORMA

Qual teste ajuda a próxima pessoa?

```
void teste1() { ... }
```

```
void pedidoFechadoNaoAceitaNovosItens() { ... }
```

O segundo nome indica o comportamento a preservar.

ATIVIDADE

ESCOLHA UMA HISTÓRIA

Como demonstrar essa regra?

| Pedido fechado não aceita novos itens.

Que estado inicial precisamos montar?

Que chamada tenta violar a regra?

Qual observação pública mostraria o resultado?

UM CENÁRIO QUE DISTINGUE RESULTADOS

O pedido fechado preserva o total

```
Produto teclado = new Produto("Teclado", 150.0);  
Produto mouse = new Produto("Mouse", 80.0);  
Pedido pedido = new Pedido();  
  
pedido.adicionarItem(teclado, 1);  
pedido.fechar(new EntregaNormal());  
pedido.adicionarItem(mouse, 1);  
  
assertEquals(150.0, pedido.calcularTotal());
```

Precisamos abrir Pedido por dentro?

O teste não lê:

- o campo `fechado` ;
- a lista privada;
- um índice ou um `if` .

Ele observa `calcularTotal()` depois da tentativa de inclusão.

ATIVIDADE

SEPARE REGRA E IMPLEMENTAÇÃO

Quando esse teste deveria quebrar?

1. Se `Pedido` trocar a forma de percorrer os itens?
2. Se aceitar o mouse mesmo depois do fechamento?
3. Ele também verifica remoção após fechar?

UMA CHAMADA SEM MUDANÇA TAMBÉM IMPORTA

Quantidade negativa preserva o item

```
Produto teclado = new Produto("Teclado", 150.0);  
Pedido pedido = new Pedido();  
  
pedido.adicionarItem(teclado, 2);  
pedido.alterarQuantidade(teclado, -1);  
  
assertEquals(300.0, pedido.calcularTotal());
```

A mesma referência de produto identifica a linha.

O EFEITO ESPERADO PODE SER NENHUM

Recusar também é comportamento

A operação foi chamada.

O estado observável deve permanecer igual.

Testar uma regra não exige que o valor final mude.

ATIVIDADE

APROFUNDAMENTO

Um total igual conta a história inteira?

Um pedido contém teclado de 150.0 e mouse de 80.0 .

Depois de uma operação, o total continua 230.0 .

Podemos concluir que cada linha permaneceu igual?

ESCOLHA A EVIDÊNCIA ADEQUADA

O cenário deve distinguir o erro

O mesmo total pode resultar de estados diferentes.

Nos nossos exemplos, cada operação proibida teria um efeito numérico visível.

Para outra regra, talvez precisemos de outro cenário ou de mais uma operação pública.

Proteja o custo que já funciona

```
@Test
void entregaNormalCustaDezReais() {
    Pedido pedido = new Pedido();
    pedido.fechar(new EntregaNormal());

    assertEquals(10.0, pedido.calcularCustoEntrega());
}
```

Quando a dependência mudar, este cenário deve continuar passando.

VAMOS PROVOCAR UMA FALHA

E se a regra for alterada por engano?

```
public double calcularCusto(Pedido pedido) {  
    return 20.0;  
}
```

O teste ainda espera 10.0 .

ATIVIDADE

PREVEJA A SUÍTE

Qual teste ficará vermelho?

- total de duas unidades de teclado;
- inclusão após fechamento;
- custo da entrega normal.

Qual diferença JUnit mostrará?

VERMELHO → VERDE

A falha identifica uma divergência

```
esperado: 10.0  
obtido: 20.0
```

Restauramos `return 10.0;` e executamos os testes outra vez.

Agora o cenário protegido passa.

O QUE ESSE VERDE DEMONSTRA?

Uma evidência delimitada

Neste cenário, o comportamento observado corresponde à expectativa codificada.

E se a expectativa estiver errada? E se faltar um cenário importante?

ATIVIDADE

DEFINA O ALCANCE DO TESTE

O que podemos concluir?

1. O teste da entrega normal passa. A entrega expressa funciona?
2. Todos os testes escritos passam. Não há mais bugs?
3. Que regra da Versão 10 ainda vale proteger?

Teste o Projeto 2 que já funciona

- contador inicial e avisos por e-mail;
- sequência com e-mail, SMS e painel;
- dois lembretes independentes;
- uma regra de contagem escolhida por você.

Qual teste você mais gostaria de ter antes de mudar **Lembrete** ?

SÍNTESE

A PERGUNTA AGORA É EXECUTÁVEL

Síntese

- Teste = cenário + operação observada + expectativa.
- Bons nomes explicam a regra protegida.
- Observamos comportamento público, inclusive quando nada deve mudar.
- Passar oferece evidência sobre os cenários escritos.

ANTES DA PRÓXIMA MUDANÇA

Podemos perguntar de novo

Comportamento atual → teste → mudança → teste novamente

Você continua fazendo aquilo que combinamos?